Universidad Autónoma del Estado de México

FACULTAD DE INGENIERÍA

PLAN DE ESTUDIOS:
Licenciatura de Ingeniería en Computación

UNIDAD DE APRENDIZAJE:
Robótica

Tarea 4

"PRUEBA DE FUNCIONALIDAD DE UN ROBOT MÓVIL DIFERENCIAL"

INTEGRANTES DE EQUIPO:
- Fernando Azuara Ibarra

GRUPO:
O1

DOCENTE:
DR. EN C.I.E. JAIME GARCÍA GARCÍA

Toluca, Estado de México, noviembre de 2026

1. OBJETIVO

Armar y verificar un robot móvil de ruedas con arquitectura diferencial (**kit LaFvin ESP32 Camera 4WD**, driver **L298N**, cuatro motores TT y baterías 18650) y programar rutinas reutilizables de movimiento (adelante, atrás, giro a la izquierda, giro a la derecha y vuelta de 360° sobre su eje). Después, **calibrar el tiempo de cada rutina** midiendo el movimiento real en el piso para saber con precisión cuánto se mueve el robot, y contrastar lo medido con el modelo cinemático diferencial.

2. MATERIALES Y DIAGRAMA DE CONEXIÓN

Se usó el kit completo con una **ESP32-CAM AI-Thinker** (cámara OV3660) programada por una placa **ESP32-CAM-MB** (CH340), el driver L298N, tres baterías 18650 (12.1 V cargadas), cinta métrica, cinta de color, multímetro y un celular para controlar y grabar. Se programó con arduino-cli y el core ESP32 3.3.12. Los dos motores de cada lado van en paralelo, por lo que el robot se comporta como diferencial y gira derrapando. La Tabla 1 muestra las conexiones y la Figura 1 las fotos del robot armado, el diagrama y la medición del voltaje.

ESP32-CAM	L298N	Función
GPIO12	IN1	Lado izquierdo, adelante (LEFT_M1)
GPIO13	IN2	Lado izquierdo, reversa (LEFT_M0)
GPIO14	IN3	Lado derecho, reversa (RIGHT_M0)
GPIO15	IN4	Lado derecho, adelante (RIGHT_M1)
5V / GND	+5V / GND	Alimentación del ESP32 desde el regulador del L298N; tierra común
Batería + / −	+12V / GND	Alimentación de motores (3×18650, por el interruptor)
—	ENA, ENB	Jumpers puestos; la velocidad se regula con PWM en los pines IN

Tabla 1. Tabla de pines (GPIO12–15 son los pines propuestos por el enunciado).

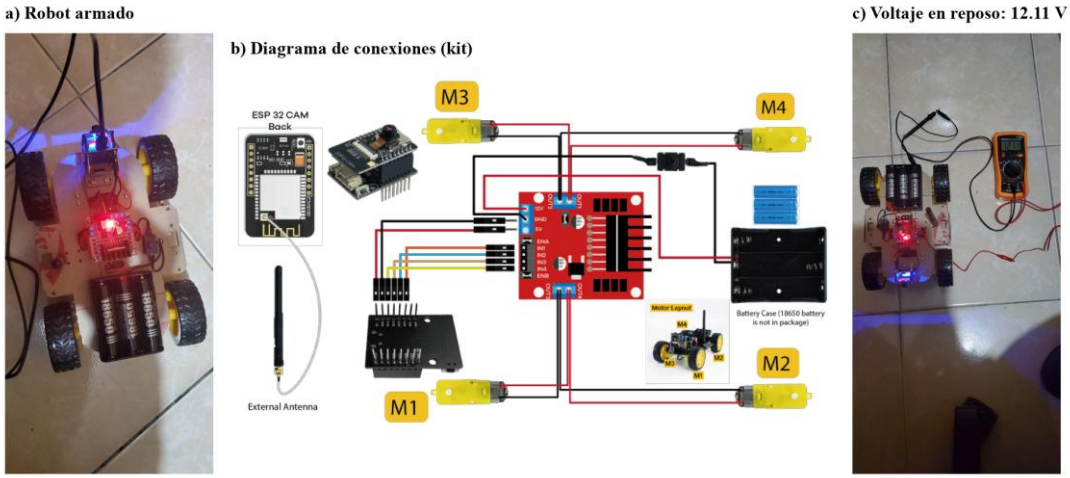


Figura 1. a) Robot armado; b) diagrama de conexiones del kit (M1–M4 son los cuatro motores); c) medición del voltaje de las baterías.

GPIO12 es un pin de arranque (strapping): si está en alto al encender, el ESP32 no arranca; con el L298N conectado arranca sin problema porque las entradas no lo jalan a alto, lo que se comprobó al energizar. Antes de mover el robot se verificó la polaridad y las tierras comunes, y se hicieron las primeras pruebas con el robot en el aire, como pide el enunciado.

3. CÓDIGO COMENTADO

El firmware está en firmware/robot_esp32cam/ (repositorio de GitHub, enlace en las conclusiones). robot_motion.cpp contiene las funciones **adelante(), atras(), izquierda(), derecha(), detener() y girar360()**, una cola de movimientos no bloqueante basada en millis() (el video y la página web siguen respondiendo mientras el robot se mueve), la secuencia de la Parte C, la prueba de PWM mínimo, la calibración guardada en flash (NVS) y los comandos por monitor serie, que imprime cada etiqueta con su marca de tiempo y la duración real medida. **La velocidad se regula con PWM en los pines IN** (2 kHz, 8 bits, canales LEDC 4–7): como los jumpers ENA/ENB están puestos, basta

enviar PWM a la entrada del sentido deseado y 0 a la otra, y el ESP32-CAM no tiene pines libres para ENA/ENB. El PWM se limita a pwm_max = 145 (≈ 6 V promedio) porque los motores TT son de unos 6 V.

```
// Un lado del robot: valor > 0 adelante, < 0 reversa, 0 detenido
static void lado(ledc_channel_t ch_ade, ledc_channel_t ch_rev, int valor) {
  if (valor >= 0) { set_duty(ch_rev, 0);      set_duty(ch_ade, valor); }
  else            { set_duty(ch_ade, 0);      set_duty(ch_rev, -valor); }
}

// Sin factor de correccion (lo usa la rampa de PWM minimo)
static void motores_crudo(int izq, int der) {
  lado(CH_IZQ_ADE, CH_IZQ_REV, limitar(izq));
  lado(CH_DER_ADE, CH_DER_REV, limitar(der));
}

// Aplica el factor de correccion de cada lado y el limite de PWM
void motores(int izq, int der) {
  motores_crudo((int)lroundf(izq * cfg.trim_izq), (int)lroundf(der * cfg.trim_der));
}

static int pwm_o_default(int pwm, int def) { return pwm < 0 ? def : pwm; }

void adelante(int pwm)  { int p = pwm_o_default(pwm, cfg.pwm_avance); motores( p,  p); }
void atras(int pwm)     { int p = pwm_o_default(pwm, cfg.pwm_avance); motores(-p, -p); }
void izquierda(int pwm) { int p = pwm_o_default(pwm, cfg.pwm_giro);   motores(-p,  p); }
void derecha(int pwm)   { int p = pwm_o_default(pwm, cfg.pwm_giro);   motores( p, -p); }
void detener()          { motores_crudo(0, 0); }
```

Figura 2. Control de bajo nivel y funciones de movimiento (`robot_motion.cpp`).

```
bool girar360() {
  return robot_mover(-cfg.pwm_giro, cfg.pwm_giro, cfg.t_giro360, "Giro 360");
}

// Parte C: adelante, atras, izquierda, derecha y 360, con 1 s de pausa
bool robot_secuencia_tarea() {
  Comando c = {};
  c.tipo = CMD_PLAN; c.n = 6;
  const int p = cfg.pwm_avance, g = cfg.pwm_giro;
  llenar_segmento(c.seg[0], "Inicio (3 s)",    0,  0, 3000,           0);
  llenar_segmento(c.seg[1], "1 Adelante",      p,  p, cfg.t_adelante, 1000);
  llenar_segmento(c.seg[2], "2 Atras",        -p, -p, cfg.t_atras,    1000);
  llenar_segmento(c.seg[3], "3 Izquierda 90",-g,  g, cfg.t_giro90,   1000);
  llenar_segmento(c.seg[4], "4 Derecha 90",    g, -g, cfg.t_giro90_der, 1000);
  llenar_segmento(c.seg[5], "5 Giro 360",     -g,  g, cfg.t_giro360,  0);
  return encolar(c);
}
```

Figura 3. Giro de 360° y secuencia de la Parte C con pausas de 1 s (`robot_motion.cpp`).

[Pendiente: captura del monitor serie con las etiquetas de la secuencia (se toma durante el video).]

4. RESULTADOS

4.1 Medidas físicas y alimentación

Se midieron tres veces las dimensiones del robot: distancia entre las llantas izquierdas y derechas **L = 12.5 cm**, distancia entre ejes **15.1 cm** y diámetro de llanta **6.5 cm**. El voltaje de las baterías fue **12.12 V en reposo** y **11.88 V con los motores girando** (caída de 0.24 V). Durante la calibración el voltaje bajó de 12.35 V a 11.83 V, así que los tiempos calibrados valen para ese intervalo.

4.2 Parte B – Prueba individual de motores

Con las llantas en el aire se probó cada lado en ambos sentidos (los sentidos son correctos sin invertir cables) y se midió el PWM mínimo con una rampa de 5 en 5 cada 0.4 s, marcando cuando **giran las dos llantas del lado**. Se alternaron lados, con 7 repeticiones por lado.

Motor	PWM mínimo (7 repeticiones)	Media ± DE	Sentido correcto	Observaciones
Izquierdo (GPIO12/13)	65 / 70 / 70 / 70 / 70 / 70 / 70	69.3 ± 1.9	Sí	Arranque consistente
Derecho (GPIO15/14)	85 / 70 / 85 / 70 / 90 / 85 / 70	79.3 ± 8.9	Sí	Arranca brusco; valores en dos grupos (70 u 85–90)

Tabla 2. PWM mínimo por lado (0–255).

El lado derecho necesita más PWM (Welch: $t = 2.92$, $p = 0.024$; Mann-Whitney: $p = 0.025$), lo que indica **más fricción estática**: según la posición en que se detiene el engranaje del motor TT basta 70 u hacen falta 85–90. Con el PWM de operación (140) ambos lados quedan muy por encima de su umbral.

4.3 Parte C – Rutinas de movimiento

La secuencia se ejecuta con el comando seq o con el botón de la página, con 3 s de espera inicial y **1 s de pausa** entre movimientos (Figura 3). La duración inicial del enunciado se calibró como muestra la Tabla 3; la página /tarea1 guía la calibración (Figura 4).

#	Movimiento	PWM izq. / der.	Inicial	Calibrada
1	Adelante	+140 / +140	2000 ms	1340 ms
2	Atrás	−140 / −140	2000 ms	1490 ms
3	Giro izquierda	−140 / +140	500 ms	480 ms
4	Giro derecha	+140 / −140	500 ms	520 ms
5	Giro 360° (opcional)	−140 / +140	1800 ms	1740 ms

Tabla 3. Duraciones de la secuencia, antes y después de calibrar.



Figura 4. Página /tarea1 del robot: E2, calibración de tiempos.

4.4 Parte D – Calibración y medición

En el piso se marcó con cinta una línea de 1 m y una cruz de ángulos. Cada duración se ajustó por regresión del tiempo contra la distancia o el ángulo medidos y se repitió hasta que dos corridas seguidas cayeran en la tolerancia. La Figura 5 muestra cómo convergió cada movimiento. Las primeras corridas del 360° dispersaban mucho porque el robot estaba frío y la primera arrancada con la rampa pierde ángulo; después de unos giros de calentamiento la repetibilidad fue de ±1 a ±5°. La Tabla 4 reúne las **últimas tres corridas** de cada movimiento.

Movimiento	Tiempo (ms)	Esperado	Medido (rep. 1 / 2 / 3)	Promedio	Error
Adelante	1325	Línea recta de 1 m	98 / 98 / 97 cm Desv. lateral: -2.0 / -2.0 / -3.0 cm	97.7 cm	-2.3 cm
Atrás	1490	Regreso al inicio	100 / 100 / 100 cm Distancia al origen (desv.): +7.5 / +3.5 / +5.5 cm	100.0 cm	+0.0 cm
Izquierda	480	90°	88 / 88 / 87°	87.7°	-2.3°
Derecha	520	90°	91 / 91 / 90°	90.7°	+0.7°
360°	1765	360°	367 / 368 / 367°	367.3°	+7.3°

Tabla 4. Parte D: últimas tres repeticiones. El tiempo cargado al final (Tabla 3) es el ajuste posterior a la última serie de adelante (1325→1340 ms) y de 360° (1765→1740 ms).

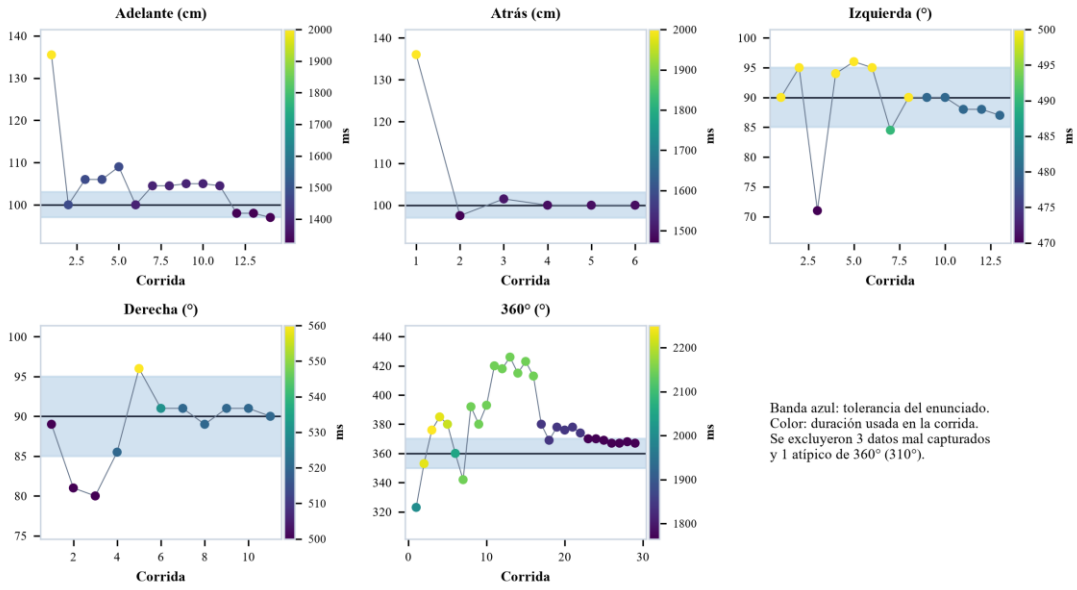


Figura 5. Convergencia de la calibración (valor medido contra número de corrida).

[Pendiente: validación con 15 repeticiones por movimiento (etapa E3) con los tiempos finales, y enlace del video de la secuencia.]

5. RESPUESTAS DEL ANÁLISIS

1. ¿Por qué se desvía en línea recta si ambos motores reciben la misma señal? El control es de lazo abierto: el mismo PWM no garantiza la misma velocidad. Influyen las tolerancias de los motores TT y sus reductoras (fricción y holgura), pequeñas diferencias de diámetro de llanta, la distribución del peso y que cada lado arranca con un PWM mínimo distinto (Tabla 2: 69 contra 79). Una diferencia de pocos por ciento produce una curvatura constante y la desviación lateral crece con la distancia. En las pruebas finales de avance de 1 m la desviación fue de -2.3 cm (con 1390 ms, antes del ajuste, llegó a +4.5 y +5 cm) y al ir en reversa llegó a ±7.5 cm, lo que confirma que el desvío cambia con la fricción de cada serie y no es una constante.

2. ¿Diferencia entre girar con un motor detenido y con motores en sentidos opuestos? Con un motor detenido, el centro de giro es la rueda detenida, $\omega = v/L$ y el robot describe un arco de radio L/2; con motores opuestos, el centro es el punto medio entre ruedas, $\omega = 2v/L$ (el doble) y el robot gira en su lugar. Este programa usa giros sobre su eje, que cambian la orientación sin desplazarse. En un 4WD ambos giros arrastran las llantas de lado, por lo que el giro real es más lento que el teórico.

3. ¿Cómo afectan la caída de voltaje del L298N y la batería? El L298N (transistores bipolares) pierde unos 2 V; el voltaje promedio del motor es ≈ PWM/255·(V_bat − 2 V). Con 12.1 V y PWM 140 son ≈ 5.5 V; con la batería baja el mismo PWM da menos velocidad, y la calibración lo mostró: con 12.1 V el avance de 1390 ms midió 104.7 cm y con 12.0 V el de 1325 ms midió 97.7 cm, y el voltaje bajó de 12.35 a 11.83 V durante los giros, lo que obligó a dejar el tiempo de 360° un poco largo. Los tiempos calibrados solo valen para ese intervalo de voltaje, y se mitiga midiendo la batería y compensando el PWM, con un driver MOSFET o cerrando el lazo con sensores.

4. Ecuaciones cinemáticas y tiempo teórico de 90°. $v = (v_R + v_L)/2$ y $\omega = (v_R - v_L)/L$. Girando sobre su eje ($v_R = v$, $v_L = -v$), $\omega = 2v/L$ y $t_{90} = \pi/(2\omega) = \pi L/(4v)$. Con $L = 12.5$ cm y $v = 70$ cm/s (distancia entre tiempo de las corridas de avance y retroceso), el tiempo teórico es **t_{90} = 139 ms**, contra **500 ms medidos**. La diferencia se explica con la **L efectiva**: con ω medida de 3.29 rad/s, $L_{ef} = 2v/\omega$ = **43 cm**, un factor de derrape L_{ef}/L = **3.4**. En un 4WD con dirección por derrape las cuatro llantas se arrastran de lado, así que el giro es mucho más lento que el del modelo ideal; la rampa de arranque y la fricción estática (más alta en el lado derecho, y por eso t90d es 40 ms mayor que t90) contribuyen. Los valores son estimaciones con las últimas tres corridas; la etapa E4 los recalcula con todos los datos.

5. Mejora propuesta. Agregar una **IMU MPU6050** para medir el ángulo real y cerrar el lazo del giro con un PID sobre la orientación (hoy el error de un giro depende del voltaje y del estado de los motores); leer el voltaje de la batería para compensar el PWM; y usar un coprocesador ESP32-WROOM-32D dedicado a motores y sensores, que libera pines para ENA/ENB, encoders y un sensor ultrasónico. La cámara del ESP32-CAM ya permite el control por WiFi con video.

6. PROBLEMAS ENCONTRADOS Y SOLUCIONES

Problema	Solución
Velocidad fija (speed = 150) y sin regulación en el código original	Funciones con PWM configurable, factor de corrección por lado y límite pwm_max
Los movimientos con delay() bloqueaban el servidor web y la cámara	Cola de movimientos no bloqueante con millis()
El servidor web admite pocas rutas por defecto	config.max_uri_handlers subido a 32
Con el core 3.3.12 la cámara no iniciaba (frame buffer malloc failed) y se detenía el arranque	camera_config_t inicializada en ceros con fb_location (PSRAM) y grab_mode; si la cámara falla, el WiFi y los motores siguen
El PWM mínimo variaba según se contara una o dos llantas del lado	Criterio fijo: se marca cuando giran las dos llantas
Calibración perdida al reiniciar	Parámetros guardados en flash (NVS); los datos de la página en LittleFS
Una batería se dañó por el pico de voltaje al conectarla con el ESP32 apagado	Conectar las baterías al final, con el ESP32 ya alimentado por USB
El 360° dispersaba mucho al inicio (240° a 426° con 2145 ms)	Calentar el robot con unos giros antes de medir, excluir el atípico y recalcular la tasa de giro
Datos mal capturados en la página (ángulos de 500°, 2050° y 1000 cm)	Se excluyeron o corrigieron y se documentaron

Tabla 5. Problemas y soluciones.

7. CONCLUSIONES

Se armó el robot, se verificaron sus motores y se calibraron sus cinco rutinas con una calibración **medida y no supuesta**: avance de 100 cm en 1340 ms, retroceso en 1490 ms, giros de 90° en 480 ms (izquierda) y 520 ms (derecha) y una vuelta de 360° en 1740 ms. Los dos giros de 90° quedaron dentro de ±5° con una desviación de 1–2° y el 360° a ±10° con una desviación de 1–4° una vez que el robot se calienta. El resultado principal es que **un robot de lazo abierto es repetible dentro de una sesión, pero no entre sesiones**: dependen del voltaje, de la fricción de cada lado y de si los motores están calientes, y por eso los tiempos solo valen para esas condiciones. El modelo cinemático ideal subestima mucho el tiempo de giro de un 4WD por derrape (L efectiva unas 3.4 veces L). Una IMU cerraría el lazo y haría innecesaria esta calibración. **[Conclusión individual: revisar y ajustar.]**

Enlaces y evidencias. Repositorio (código, datos y videos): https://github.com/igream/ESP32-AI-Robot · Video de la secuencia: **[pendiente]**. Evidencias incluidas: foto del robot armado y del diagrama (Figura 1), capturas del código (Figuras 2 y 3), tablas de calibración (Tablas 2–4) y datos crudos en docs/Tarea1/tarea1_20261109031058.json.

REFERENCIAS

[1] STMicroelectronics, "L298: Dual full-bridge driver," hoja de datos.
[2] Espressif Systems, "ESP32 Series Datasheet."
[3] Curso de Robótica, UAEMéx, "Tarea: Prueba de funcionalidad de un robot móvil diferencial," enunciado, 2026.